CODER — User Manual

CODER is a programmable module: you type a short program into the module, and the program becomes the patch. Step sequencers, effects, modulation sources, and small realtime DSP structures are all written the same way — a few plain lines. No programming experience is assumed: if you can patch a modular, you can read every example in this manual.

This is the manual for the **rescue build (2026)**. CODER originally shipped in 2019 with a JavaScript engine inside; that engine is gone. In its place is a small language designed for modular work. Programs are short — most of this manual's examples are one line — and the module is built to be forgiving: **a typo never stops the sound.** If your program has a mistake, the last good program keeps playing and the error is written on the panel, in plain words, with its line number.

Quick start

A fresh CODER already contains a starter program. It plays immediately — CODER keeps its own time when no clock is patched:

```
# SETUP
let seed = 1
let tempo = 120

# PLAY
let bass = semi([0 0 12 0 3 0 10 7])
out1 = bass
out2 = gate(spread(3, 8))
```

- **OUT 1** carries a little bassline (stepped pitch CV).
- **OUT 2** fires gates in a 3-against-8 pattern.
- Patch OUT 1 to a VCO's V/OCT input and OUT 2 to an envelope's gate input and you have a voice. Patch a clock into **IN 1** and CODER follows it instead of its internal tempo.

Your first one-line program, in the PLAY box, is just:

```
out1 = in1
```

—a patch cord. Everything else in this manual is a variation on that line.

The two boxes

SETUP (top) runs **once**, when your program loads. Definitions live there: patterns you want to name, the tempo, the random seed, the clock choice.

PLAY (bottom) is the live program: the lines that wire outputs.

Both boxes use the same language. Any `let` definition may live in either box — SETUP is simply where the shared ones belong. Only two things are box-specific: outputs are wired in PLAY (an `out` line in SETUP is an error), and the SETUP directives (`tempo`, `clock`, `seed`) only count in SETUP. Editing either box recompiles the whole program; the new program takes over only if it is completely correct.

Lines

One statement per line. A line is one of:

- `let name = ...` — define something and give it a name.
- `out1 = ...` (also `out2`, `out3`, `out4`; the old spellings `output1` … `output4` still work) — wire an output jack.

`#` starts a comment, anywhere on a line.

Values

Programs work with three kinds of values:

- **Numbers** — plain values: `0.5`, `5`, `-2.3`. In this module, numbers are volts unless a construct says otherwise.
- **Signals** — streams that flow per sample: jack inputs, oscillators, filter outputs. Signal math is continuous: `in1 * 0.5` is a VCA.
- **Patterns** — looping lists of steps. A pattern used in a line reads as its current step's value, held until the next step — stepped CV.

The clock

Patterns advance on **the clock**: the gate on IN 1 by default. CODER chooses by the **cable**: if the clock input has a cable plugged into it, CODER follows that jack; if not, CODER uses its internal clock (120 BPM, or whatever `let tempo = ...` says in SETUP) — even if a different jack is carrying a clock signal. To move the clock: `let clock = in2` in SETUP. Patterns of different lengths share the clock, so polymeter is automatic. `phase` reads 0→1 inside the current step; `beat` counts steps.

Errors can never hurt you

- A bad line changes nothing audible. The previous good program keeps running; the Error LED lights and the message appears on the status line at the **top of the panel**,

next to the LED: `Line 3: I don't know 'lp2()'. Did you mean 'lp()'?` (Hover the LED or the message to read a long message in full.)

- There are no loops and no way to write a program that hangs: a program's size is capped, and every value is sanitized as it flows (see Limits).

Language reference

This is the complete reference for the language — every word the compiler knows, with its exact behaviour. Every construct has an example you can paste into PLAY as-is. Unless noted, examples assume nothing is in SETUP.

How to read the entries. Each word is shown as `word(argument, …)`. Where an argument is called `x` it is the signal being processed. Almost every argument may be any expression — a number, a named value, a jack input, a whole nested phrase — so most controls can be modulated by CV. The few arguments that must be **plain numbers** (written literally, not wired to a signal) are marked **[number]**. The language is **case-sensitive**: every built-in word is lowercase, and `Bass` and `bass` are different names.

Word index

Group	Words
Built-in values	`in1..in4` `gate1..gate4` `trig1..trig4` `sr` `beat` `phase`
Patterns	`[ … ]` `spread` `gate` `flip` `shift` `thin` `slow` `fast` `pick`
Pitch	`quant` `semi`
Filters & delays	`lp` `hp` `reso` `echo` `tap` `z`
Shaping	`drive` `fold` `crush` `clip`
Conditioning	`slew` `sah` `env` `follow`
Sources	`lfo` `noise` `rand` `walk` `loop` `cycle`
Math	`+ - * / %` `abs` `sin` `min` `max` `clamp` `map` `range` `mix`
SETUP directives	`tempo` `clock` `seed`

Syntax: lines, names, numbers

- One statement per line; `#` comments out the rest of a line.
- A **name** starts with a letter or `_`, followed by letters, digits and `_`. A name must be defined on a line above the lines that use it, and cannot be defined twice. CODER's own words are reserved — you cannot name something `lp`, `in1`, `beat`, `let`, or any other word from the index.

- **Numbers** are written plainly: `5`, `0.5`, `.5`, `-2.3`. There is no scientific notation (`1e3` is not a number here), and a malformed number like `1.2.3` is rejected with an error.
- Precedence, tightest first: brackets and calls; unary minus; `* / %`; `+ -`; and loosest, the patch-cord arrow `->`. Parentheses override, as usual.

Inputs

Name	Meaning
`in1` … `in4`	the raw voltage on each input jack, per sample
`gate1` … `gate4`	the same jacks as gates: **1** while the jack is above 0 V, else 0
`trig1` … `trig4`	a **5 V** pulse, exactly one sample long, on the sample the jack rises past 0 V
`sr`	the sample rate
`beat`	how many clock steps have passed since the module loaded
`phase`	0→1 position inside the current clock step

Note the two gate levels: `gate1` reads a jack as 1/0 logic, while the `gate(...)` word writes full 5 V pulses. Triggers inside the language (`env`, `sah`) fire when their trigger signal rises past **1 V**.

```
out1 = in1 * gate2          # a VCA opened by jack 2
out1 = phase * 5            # a ramp that restarts every clock step
```

Wiring and naming

```
let bass = semi([0 0 12 0]) # name anything: a pattern, a signal, a number
out1 = bass                 # wire a jack
out2 = in1 * 0.5            # or wire an expression directly
out3 = 2.5                  # a bare number is a constant voltage
```

Each output may be wired **once** — wiring `out1` twice is an error. An output you never wire sits at 0 V.

The patch-cord arrow: `->`

Nested brackets read inside-out, which is nobody's favorite. The arrow `->` lets a line read left to right, the way a patch cord runs: whatever is on the left flows into the first input of the word on the right.

```
out1 = in1 -> lp(900) -> drive(0.3)
```

is exactly `drive(lp(in1, 900), 0.3)`, written the way you patch it. Arrows chain, and they bind looser than arithmetic: `in1 + in2 -> lp(900)` filters the sum. The arrow supplies the word's first argument, whatever kind it is; words that take no input of their own (`noise()`, `rand()`) refuse an arrow with an error.

The clock, exactly

- Patterns, and the step-driven sources (`rand`, `walk`, `loop`, `cycle`), advance once per clock step.
- **External clock** (the clock jack has a cable in it): a step happens on each rising edge of the jack — the moment it crosses above 0 V. `phase` measures where you are between the last two edges, as a 0→1 fraction; if the clock pauses, `phase` sits at 1 until the next edge.
- **Internal clock** (no cable in the clock jack): one step per beat at the SETUP tempo — `let tempo = 132` — which must be between 20 and 400 and defaults to 120. `phase` sweeps 0→1 across each step.
- `beat` counts steps. It starts when the module loads and does **not** restart when you edit — but a successful edit does restart the patterns at step 1 and empties filter and delay state, so expect a small hiccup in sustained sounds when you change code mid-performance.

Patterns

A pattern is a bracketed list of steps. Values are volts. A `-` standing alone is a **rest**; a minus stuck to a number is a negative number (`[0 -2]` steps 0 then −2; `[0 - 2]` steps 0, rest, 2). A `_` is a **tie**: the step holds — a pitch stays, and a gate stays high through the tied step instead of re-pulsing, so `[1 _ 0 0]` is one long gate, not two. Patterns can be up to 64 steps, and must have at least one.

Reading a pattern as a value. A pattern in an expression reads as its current step's value, held between steps. A rest or tie step has no value of its own: it reads as the previous sounding step's value, so `[0 - 5]` reads 0, 0, 5 — rests are silent only to `gate(...)`, below. If a pattern starts with rests, it reads its first sounding value from the very beginning, until a real step sounds.

```
out1 = semi([0 0 12 0 3 0 10 7])   # write melodies in semitones with semi()
out1 = [0 0 2.5 0]                 # or in raw volts
```

`gate(p)` — a pattern as gates. Emits 5 V pulses. The exact rule: a step produces a gate only if it is **not a rest and its value is above 0** — so zero-valued steps are silent, exactly like rests. The pulse is high for the first half of the step, and if the next step is a tie (`_`), the gate stays high through that whole tied step too (and through a run of ties).

```
out2 = gate(spread(3, 8))          # rhythm patterns: hits are 1s
out2 = gate([0 0 12 0])            # fires ONCE, on the 12 step — the 0s are silent
```

Pattern verbs. Each returns a new pattern; the original keeps playing its own course, so a pattern and its transforms all advance in lockstep. The verbs work on `[ ]` patterns and on `spread(...)` results (and on other verbs' results) — they cannot be applied to a `pick(...)` pattern; that is a compile error.

Verb	Meaning
`spread(hits, steps)`	distribute hits as evenly as possible over the steps. **[number]** ×2: steps 1–64, hits 0–steps. Hits are 1s, the rest are rests — `spread` is a gate pattern: use it through `gate(...)` (read as a value it is simply 1)
`flip(p)`	play the pattern backwards
`shift(p, n)`	rotate by `n` steps: positive `n` moves every step later, negative earlier. `n` is a **[number]**
`thin(p, amt)`	each step independently becomes a rest with probability `amt` (0–1, a **[number]**). The choice is made once, at compile time, from the seed — same program, same thinning, every load
`slow(p, n)`	each step lasts `n` clocks. `n` is a **[number]**, 1–32; the result must still fit in 64 steps
`fast(p, n)`	keep every `n`th step (steps 1, 1+n, 1+2n, ...), so the pattern cycles `n` times as fast. `n` is a **[number]**, 1–32
`pick(p, p, …)`	two or more patterns; plays one of them, and chooses again — at random, from the seed — each time the first pattern's length completes. Choices may have different lengths; all advance together

```
out1 = gate(flip([1 0 0 1]))       # flip(p): play it backwards
out1 = gate(shift([1 0 0 0], 1))   # shift(p, n): the hit moves to step 2
out1 = slow([0 7], 2)              # slow(p, n): each step lasts n clocks
out1 = fast([0 3 5 7], 2)          # fast(p, n): steps 0 and 5, twice as fast
out1 = gate(thin([1 1 1 1], 0.5))  # thin(p, amt): drop steps at random (seeded)
out1 = pick([0 0 12 0], [0 5 7 0]) # pick(p, p, …): a fresh choice each cycle
out2 = gate(spread(3, 8))          # gate(p): 5 V pulses (see the rule above)
```

Pitch

```
out1 = quant(in1)                  # snap a voltage to semitones (1 V/oct)
out1 = quant(in1, minor)           # or to a scale: major, minor, penta, dorian
out1 = semi(7)                     # semitones to volts: semi(n) = n / 12
```

- `quant(x)` snaps to the nearest semitone. An exact half-semitone rounds **up**.
- `quant(x, scale)` snaps to the nearest note of the scale. The scales are anchored at **C** (0 V = C, always — they are pitch classes, not relative to your signal): major = C D E F G A B; minor = C D E♭ F G A♭ B♭; penta = C D E G A; dorian = C D E♭ F G A B♭. When two allowed notes are equally near, the **lower** one wins.
- `semi(x)` divides by 12. On a number or signal it converts semitones to volts; on a **pattern** it returns a new pattern with every step divided by 12 — which is how the examples in this manual write melodies.

Effects and signal shaping

Effects are words. Each keeps its own state inside the module; chain them with ordinary lines. Frequencies are in Hz, times in seconds.

Word	Behaviour
`lp(x, hz)`	one-pole lowpass. `hz` is clamped inside to 1 Hz – just under half the sample rate
`hp(x, hz)`	one-pole highpass, same clamping
`reso(x, hz, q)`	resonant lowpass (state-variable). `hz` is clamped to 5 Hz – 1/6 of the sample rate (about 8 kHz at 48 kHz). Higher `q` rings more; from about `q = 20` upward it is at maximum ring, and the internals are clamped so it cannot run away
`echo(x, sec, feedback, mix)`	feedback delay. `sec` is clamped to 1 ms – 2 s; `feedback` to −0.98 – 0.98 (negative feedback inverts each repeat); `mix` 0–1 blends dry and delayed: output = dry·(1−mix) + delayed·mix
`drive(x, amt)`	soft saturation: a tanh curve, gain rising with `amt` (0–1). Its output always lives within **±1 V** — follow it with a `* 5` if you need level back
`fold(x, amt)`	wavefolder. The signal is scaled up with `amt` (0–1) and folded back on itself; output is bounded to ±5 V
`crush(x, amt)`	bit + sample-rate reduction. `amt` 0 is a clean passthrough; as `amt` rises to 1 the bit depth falls 16 → 2 bits and the hold time grows to 47 samples
`clip(x, lo, hi)`	hard limits: the signal is held inside `lo`–`hi`. Exactly the same as `clamp` (below) — the two words are synonyms

```
out1 = lp(in1, 900)                       # lp(x, hz): one-pole lowpass
out1 = hp(in1, 300)                       # hp(x, hz): one-pole highpass
out1 = reso(in1, 800, 6)                  # reso(x, hz, q): resonant filter
out1 = echo(in1, 0.375, 0.35, 0.3)        # echo(x, sec, feedback, mix)
out1 = drive(in1, 0.5)                    # drive(x, amt): soft saturation, amt 0..1
out1 = fold(in1, 0.4)                     # fold(x, amt): wavefolder
out1 = crush(in1, 0.6)                    # crush(x, amt): bit + sample-rate reduction
out1 = clip(in1, -2, 2)                   # clip(x, lo, hi): hard limits
```

Filter frequencies (and most other arguments) can themselves be signals:

```
out1 = lp(in1, 400 + lfo(0.3, sine) * 60)  # a filter that breathes
```

Realtime DSP from primitives

`tap` reads a signal delayed; `z` is a signal's value **one sample ago**. A signal may refer to itself **only** through `z(...)`, or through the first argument of `tap(...)` — that single rule is what makes feedback safe to write, and it is enough to build real DSP structures from raw algebra. Self-reference anywhere else — including inside `echo(...)` — is a compile error that tells you so.

- `z(x)` — `x` one sample ago; before the first sample it reads 0. Nest it for longer sample delays: `z(z(x))` is two samples ago.
- `tap(x, sec)` — `x` delayed by `sec` (clamped 0 – 2 s). Reads are interpolated, so modulating `sec` glides smoothly instead of stepping. A delay under one sample passes the signal through unchanged.

```
let y = in1 * 0.2 + z(y) * 0.8            # a one-pole filter, by hand
out1 = y
```

```
# a lowpass written out as a biquad (1000 Hz, Q 0.71, at 48 kHz)
let y = 0.00392*in1 + 0.00783*z(in1) + 0.00392*z(z(in1)) + 1.8154*z(y) - 0.8310*z(z(y))
out1 = y
```

```
let comb = in1 * 0.5 + tap(comb, 0.011) * 0.6   # a feedback comb, by hand
out1 = comb
```

Delays run up to 2 seconds. Feedback amounts are clamped inside the runtime, so a structure like this can never run away.

Conditioning and modulation

Word	Behaviour
`slew(x, sec)`	glide: the output moves toward the input at a fixed rate — `sec` is the time to swing a full 5 V. `sec` 0 (or less) is instant
`sah(x, trigger)`	sample & hold: when `trigger` rises past 1 V, the output takes `x`'s current value and holds it until the next rise. Starts at 0
`env(trigger, attackSec, decaySec)`	attack/decay envelope, 0–5 V. On a rise past 1 V it attacks to 5 V in `attackSec` (minimum 0.5 ms), then decays exponentially with time-constant `decaySec` (minimum 1 ms). Retriggering restarts the attack from the current level
`follow(x, sec)`	envelope follower: tracks the level of `x` (its absolute value) with time-constant `sec` (minimum 1 ms)
`lfo(hz, shape)`	low-frequency oscillator, ±5 V. `hz` is clamped to 0–4000. `shape` is a plain word: `sine`, `tri`, `saw`, or `pulse` (a ±5 V square)
`noise()`	white noise, ±5 V, a fresh value every sample. Seeded: the same stream every load

```
out1 = lfo(2, sine)                # lfo(hz, shape): sine, tri, saw, pulse — ±5 V
out1 = noise()                     # white noise, ±5 V (seeded: same every load)
out1 = slew(in1, 0.2)              # slew(x, sec): glide — 0.2 s to swing 5 V
out1 = sah(in2, trig1)             # sah(x, trigger): sample & hold
out1 = env(gate(spread(2, 8)), 0.01, 0.4)   # env(trigger, attackSec, decaySec): 0–5 V
out1 = follow(in1, 0.05)           # follow(x, sec): envelope follower
```

Step-driven sources

These move **on the clock**: each one changes value when a step happens, then holds. All three read 0 until the first step.

Word	Behaviour
`rand()`	a new random voltage, 0–5 V, every clock step (seeded)
`walk(amt)`	a random walk: every step it moves by `+amt` or `−amt` at random, clamped inside ±5 V. `amt` may be a signal
`loop(steps, lock)`	a Turing-style looping sequence: `steps` random values (0–5 V), played in a loop. `steps` is a **[number]**, 2–64. Each time the loop comes around, every step re-rolls with probability (1 − `lock`): `lock` 1 = frozen forever, 0 = a brand-new sequence every cycle. `lock` may be a signal

Word	Behaviour
`cycle(a, b, …)`	a sequential switch: steps through its inputs, one per clock step, starting on the first. Needs at least two inputs. All inputs are evaluated every sample — their own state keeps flowing; the switch only chooses which one is heard

```
out1 = rand()                      # a new random voltage (0–5) every clock step
out1 = walk(0.4)                   # a random-walk voltage, moves on every clock step
out1 = loop(8, 0.85)               # loop(steps, lock): a random sequence that repeats;
                                   # each cycle, steps re-roll with chance (1 − lock)
out1 = cycle(in1, in2, in3)        # a sequential switch on the clock
```

Two idioms worth knowing:

```
out1 = in1 * clamp(1 - follow(in2, 0.3) / 5, 0, 1)   # ducking: in2's loudness turns in1 down
out2 = gate(slow([1], 4))                            # a clock divider: one gate every 4 steps
```

Math

`+ - * / %` and parentheses. `/` and `%` by zero give 0 (never an error, never infinity). `%` is the remainder of a division. Plus:

Word	Behaviour
`abs(x)`	absolute value — full-wave rectification
`sin(x)`	sine **in cycles**: `sin(0.25)` = 1. One full cycle per 1.0 of input
`min(a, b)` / `max(a, b)`	the lower / higher of two values
`clamp(x, lo, hi)`	hold `x` inside `lo`–`hi`. Exactly the same as `clip` — the two words are synonyms
`map(x, a, b, c, d)`	rescale: `x` at `a` gives `c`, at `b` gives `d`, linearly in between (and beyond). If `a` equals `b`, the result is `c`
`range(x, lo, hi)`	rescale the standard −5..5 range to `lo`..`hi` — `map` with the input range already filled in
`mix(a, b, t)`	crossfade: `t` 0 gives `a`, 1 gives `b` (`t` is clamped to 0–1)
`semi(x)`	semitones to volts, `x / 12` — see Pitch

```
out1 = abs(in1)                    # rectification
out1 = min(in1, in2)               # the lower of two signals
out1 = max(in1, 0)                 # half-wave rectification
out1 = clamp(in1, -1, 1)           # keep a signal inside bounds
out1 = range(in1, 0, 10)           # rescale the standard −5..5 range to 0..10
out1 = map(in1, -5, 5, 0, 10)      # rescale any range to any other, explicitly
out1 = mix(in1, in2, 0.25)         # crossfade, t 0..1
out1 = sin(phase)                  # sin() works in cycles: sin(0.25) = 1
```

SETUP directives

In SETUP, three `let` lines are also read as settings for the whole program. They only count in SETUP — the same lines in PLAY are ordinary definitions and change nothing.

```
let tempo = 132        # internal clock speed, 20–400 (default 120);
                       # used when no clock cable is patched
let clock = in2        # take the pattern clock from another jack
                       # (must be exactly in1, in2, in3 or in4)
let seed = 7           # the random seed: same seed + same program = same music
```

`seed` must be written as a **plain number**, exactly as above. The default seed is 1. The seed drives everything random in the program: `noise`, `rand`, `walk`, `loop`, `thin`, and `pick`.

Errors: what is checked while you type

CODER compiles on every edit, and the compiler checks the whole program — not just the line you touched. SETUP is compiled first, so a SETUP error is the one you see until it's fixed. Only the **first** error is shown: on the status line at the top of the panel, next to the red Error LED, as `Line 3: …` — or `SETUP Line 2: …` for the SETUP box (line numbers count within their own box). Hover the LED or the message for the full text of a long message. A failed compile changes nothing audible: the last good program keeps playing.

The checks include:

- **Unknown words and names**, with a did-you-mean suggestion when you're close: `I don't know 'lp2()'. Did you mean 'lp()'?`
- **Reserved words** used as names, and **names defined twice**.
- **Wiring mistakes**: an output wired twice, or an `out` line in SETUP.
- **Wrong kinds of argument**: a pattern where a signal is needed, a signal where a plain **[number]** is required, a scale or shape word that isn't one of the choices.
- **Broken syntax**: missing `=`, commas or closing brackets; extra tokens after a complete statement; characters the language doesn't use; malformed numbers.

- **Sizes and ranges**: an empty pattern, a pattern over 64 steps, `spread` hits outside 0–steps, `loop` outside 2–64 steps, `slow`/`fast` factors outside 1–32, a `slow` result over 64 steps, `tempo` outside 20–400, `clock` set to something other than an input, a program over 512 parts.
- **Feedback discipline**: a signal naming itself anywhere except inside `z(...)` or the first argument of `tap(...)`.

Every message is written to be acted on — it names the line, says what it expected, and shows the shape of a correct line.

Tutorial: first sound to a full patch

1. One line — a gated tremolo. Patch audio into IN 1, a gate into IN 2:

```
out1 = in1 * gate2
```

2. Two lines — give it movement.

```
out1 = lp(in1, 700 + lfo(0.25, sine) * 80) * gate2
```

3. Three lines — CODER becomes the sequencer. Unpatch the gate; patch OUT 2 to your envelope instead:

```
let bass = semi([0 0 12 0 3 0 10 7])
out1 = bass
out2 = gate(spread(3, 8))
```

Patch OUT 1 to a VCO's V/OCT and OUT 1's voice audio back into IN 1, and step 2's filter line now processes your sequence.

4. A full patch (10 lines). A small self-playing system: bassline, hats made of shaped noise, a wandering modulation, and an echoed copy of the bass voice (patch the voice's audio into IN 1):

```
# SETUP
let seed = 3
let tempo = 132

# PLAY
let bass = semi([0 0 12 0 3 - 10 7])
let hats = spread(5, 8)
out1 = quant(bass, minor)
out2 = gate(spread(3, 8))
out3 = env(gate(hats), 0.002, 0.05) * noise() * 0.2
out4 = echo(lp(in1, 900), 0.341, 0.35, 0.3)
```

Every line of that is a patching idea you already know, written down.

Limits and guarantees (the containment law)

- **Size:** a program may use at most 512 parts (each word like `lp(...)` is a part). Patterns: at most 64 steps. Delays: at most 2 seconds each. Measured on a typical machine, ordinary programs — sequencers, effect chains, hand-built filters — use about 1% of one processor core. The expensive parts are `reso(...)` and `sin(...)`: chaining a hundred of those in a row will eat most of a core, so keep heavy parts in the dozens. The 512-part cap is a safety bound, not a performance promise.
- **Safety:** every flowing value is sanitized — not-a-number and infinity become 0; outputs are clamped to ±10 V; feedback states that decay toward nothing are flushed to exactly zero (no denormal slowdowns). A badly behaved program degrades to silence on the offending part — never a runaway, never a crash, never a frozen host.
- **Determinism:** randomness is seeded. Same program + same seed + same inputs ⇒ the same output, every load. Patterns restart at step 1 when a preset loads.
- **Presets:** your program text (both boxes) is saved with the preset and restored with it.
- **Sample rate:** `sr` and all delay/filter math use the host's rate at compile time — editing the program re-reads it.
- A signal only flows while it is wired to something (directly or through a `let` that is). Unused definitions cost nothing.

Coming from the old JavaScript CODER?

The old box ran JavaScript on gate edges. **Old scripts do not run in this build** — deliberately: the JavaScript engine was the fragile part, and it is gone. The translation is small, because the new language keeps the old names where it counts:

Old (JavaScript)	Now
`output1 = gate1 * 5`	`out1 = gate1 * 5`
variables declared in Globals	`let` definitions in SETUP
`lastgate1` edge checks	`trig1` … `trig4`, or `env(...)` / `sah(...)`
(not possible) analog input values	`in1` … `in4`
`output1` … `output4`	still accepted — or write `out1` … `out4`

The old module also ran your script only when a gate changed. The new one runs continuously — which is why effects and audio-rate DSP are possible at all.

Changelog — rescue build (2026)

- New language and runtime replace the JavaScript engine (see above).
- Errors are shown on the panel in plain words, with line numbers; a bad program never interrupts the current sound.
- Programs pre-load with a commented starter example; editor boxes use a monospace font; boxes are captioned SETUP / PLAY; jacks IN 1–4 / OUT 1–4.
- Program text is saved in presets explicitly, and restored on load.
- Full manual published with the module.

Manual revision (2026-11-07)

- The language reference is now complete: every word has a full entry with its argument ranges and exact behaviour, including the `gate(...)` rule, pattern reading rules, feedback rules, and the full list of edit-time checks. Two corrections from the first edition: the error readout is the status line at the **top** of the panel (not under the PLAY box), and quantize tie-breaking is stated per mode (`quant()` rounds half-semitones up; scale mode chooses the lower note).

Acknowledgments

The design of this language was informed by the live-coding tradition — TidalCycles' patterns-as-values, Sonic Pi's plain-language teaching, and ChucK's respect for musical time are acknowledged with thanks. The language, its vocabulary, and this implementation are original to BenDavis.IO; no syntax or code was taken from any of them.